



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Desenvolvimento de Aplicações Móveis
Relatório: Projeto Final

Turma: 61D

Nome: Miguel Alcobia

Número: 50746

Professor: Pedro Fazenda

Data: 10 de Junho de 2025

Índice

Lista de Figuras	3
Introdução	4
Concepção	5
Público Alvo	5
Nome e Descrição Geral	5
Funcionalidades	5
Data Model	6
Wireframes e <i>Mockups</i>	6
Modelo Entidade Associação	7
APIs	8
Pré-produção	8
Estrutura e fundamentos da aplicação	8
Produção - Implementação	10
Sistema de Registo e Login	10
Navegação	11
API Client	12
Ecrã Principal	13
Lista de Filmes	14
Perfil de Filmes e Personagens	14
Perfil do utilizador	15
Favoritar Filmes e Personagens	16
Criar Listas	17
Mudar Cartazes dos Filmes	18
Conclusão	19
Bibliografia	20

Lista de Figuras

Figura 1 - Wireframes da app	6
Figura 2 - Mockups	7
Figura 3 - Modelo Entidade Associação	7
Figura 4 - Estrutura do User no Firestore Database	8
Figura 5 - Estrutura das listas criadas pelo User	9
Figura 6 - Ecrã de Login	11
Figura 7 - Ecrã de Criar Conta	11
Figura 8 - Endpoint para o obter os posters.	12
Figura 9 - Ecrã Principal	13
Figura 10 - Ecrã com Lista de Filmes	14
Figura 11 - Ecrã do Perfil do Filme	15
Figura 12 - Ecrã do Perfil do Personagem	15
Figura 13 - Estrutura dos dados do Firebase, associados ao user	16
Figura 14 - Ecrã do Perfil do User	16
Figura 15 - Estrutura das listas criadas pelo utilizador	17

Introdução

A aplicação que o aluno escolheu realizar trata-se de um catálogo onde o utilizador pode pesquisar por filmes e personagens da Disney, marcar os seus favoritos e criar listas com filmes para ver mais tarde.

Ao longo do relatório serão expostas as várias fases do projeto: Concepção, Pré-Produção, Produção e Pós-Produção.

Serão apresentadas as várias funcionalidades a implementar e as soluções encontradas para que estas fossem realizadas com sucesso com o mínimo de problemas possíveis.

A decisão de fazer uma aplicação sobre filmes e personagens da Disney surgiu, por usar um site com uma finalidade semelhante para banda desenhada japonesa (MyAnimeList).

Concepção

Público Alvo

O público alvo são utilizadores que gostam dos filmes da Disney e dos personagens que lhes dão vida. Com esta aplicação poderão organizar melhor as maratonas que pretendem fazer e catalogar os seus filmes e personagens preferidos.

Embora seja possível fazer algo parecido em aplicações como o IMBD, não permite ao utilizador ter um controlo específico sobre ver quantos filmes já viu de uma dada lista e não consegue indicar os seus personagens favoritos e quais filmes este entra

Nome e Descrição Geral

A *app* a desenvolver terá o nome **Disney Journal** e visa permitir o utilizador a pesquisar e favoritar os seus personagens e filmes preferidos da Disney. Além disso também permite o utilizador fazer listas com vários filmes e acompanhar o progresso da lista à medida que vai assistindo a esses mesmos filmes.

A maior inspiração para esta *app* foi o *site* MyAnimeList, que é um site para catalogar e avaliar mangás (banda desenhada japonesa) e animes (desenhos animados japoneses).

Funcionalidades

A aplicação apresentará as seguintes funcionalidades:

- **Pesquisar personagem:** O utilizador poderá pesquisar o nome de um personagem na barra de pesquisa.
- **Pesquisar filmes:** O utilizador poderá pesquisar o nome de um filme na barra de pesquisa.
- **Favoritar filmes:** O utilizador poderá favoritar um ou mais filmes que goste.
- **Favoritar personagens:** O utilizador poderá favoritar um ou mais personagens que goste.
- **Criar uma lista de filmes:** O utilizador poderá criar listas personalizadas
- **Adicionar/Remover filmes de uma lista:** Com a lista criada, o utilizador pode remover e adicionar filmes nela.

- **Acompanhar o progresso de uma lista:** Ao assistir um filme e marcá-lo como visto, a barra de progressos, que tem como valor total a quantidades de filmes da lista, incrementa.
- **Serviço Premium:** Ao ter acesso *premium* o utilizador pode criar mais do que 5 listas e também poderá personalizar as capas dos filmes com imagens à sua escolha.

A existência da funcionalidade *premium* de mudar os cartazes dá-se pelo facto dos cartazes utilizados serem os cartazes originais da época de lançamento de cada filme. Logo, estes mesmos cartazes podem não ser muito atrativos e interessantes para a maioria dos utilizadores.

Data Model

- **Movie** - name, date, posterImage, time, resume, isFavorite, customPoster
- **Character** - name, movies, profileImage, isFavorite
- **listMovies** - nameList, progress, totalTime, movies
- **listCharacter** - characters
- **User** - name, listMovies, listCharacters, premium

Wireframes e Mockups

A partir das *features* descritas anteriormente, desenharam-se as seguintes wireframes e *mockups*.

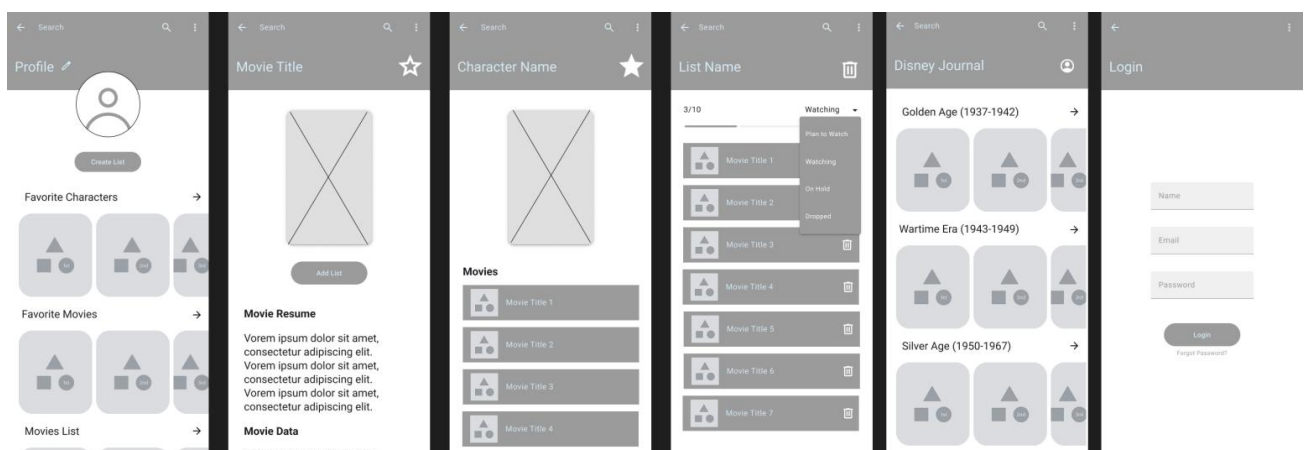


Figura 1 - Wireframes da app

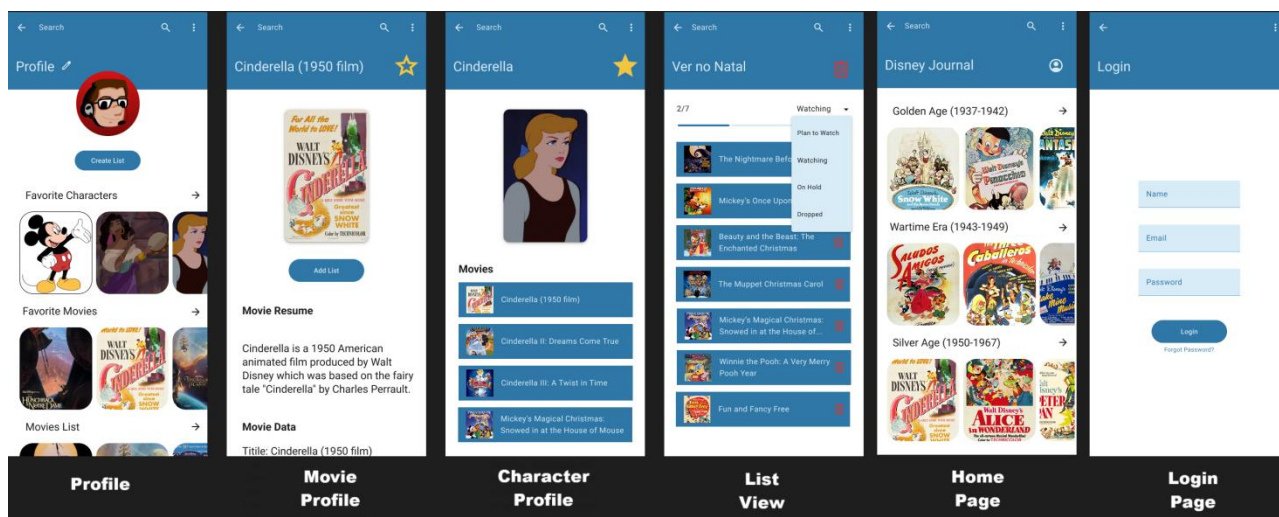


Figura 2 - Mockups

Procurou-se fazer uma UI simples e minimalista de certa forma, mas que ainda assim pudesse atender todas as funcionalidades da melhor forma. A paleta de cores, foi inspirada na personagem Cinderella que é umas das “caras” da marca Disney.

Modelo Entidade Associação

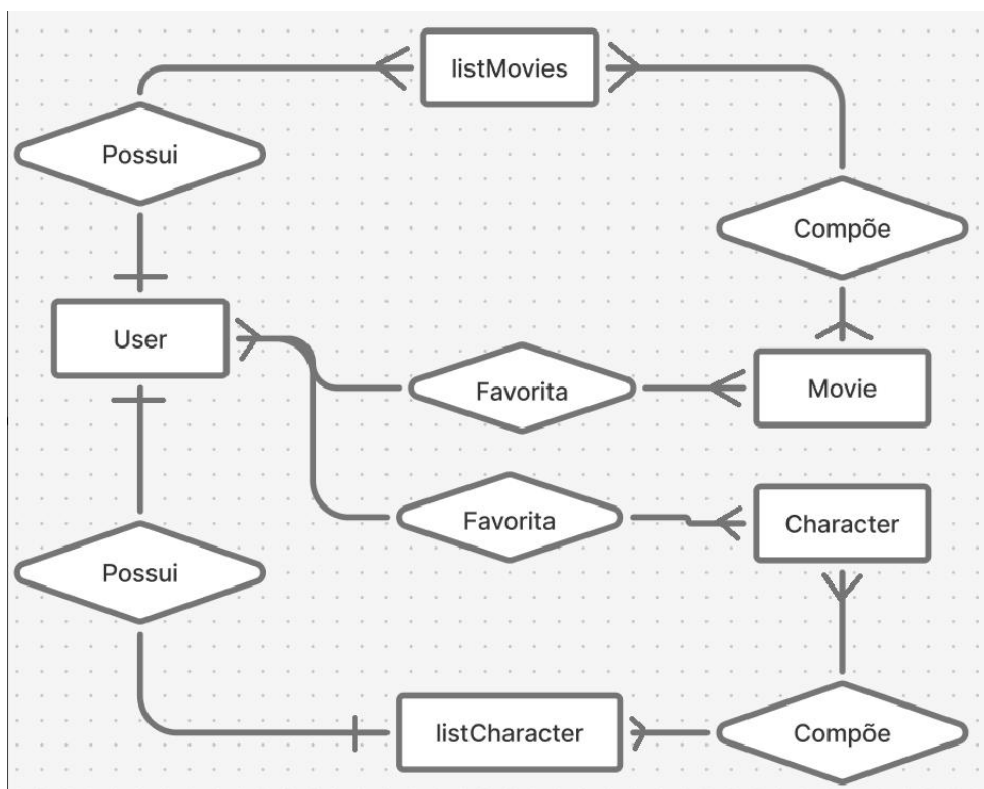


Figura 3 - Modelo Entidade Associação

APIs

Usaram-se as seguintes APIs para auxiliar na realização do trabalho:

- <https://disney.fandom.com/api.php> - API da Disney WikiFandom, com objetivo de obter os pôsteres e informações dos filmes da Disney.
- <https://disneyapi.dev/> - API aberta que, a partir da Disney WikiFandom, oferece informações, de diversos tipos, sobre os vários personagens da disney.

Pré-produção

Estrutura e fundamentos da aplicação

A aplicação obrigará o login do utilizador assim que este entre, devido a que as listas e favoritos que o utilizador criar têm de ser devidamente guardados e carregados no arrancar da aplicação. O login e a gestão de utilizadores fica a cargo do Firebase, que também será responsável por guardar em JSON, as listas de filmes e de personagens favoritos do utilizador.

Para os pedidos da API será usado Ktor e também o Coil para ajudar no processar dos URLs das imagens fornecidas pelas APIs.

No Firebase, mais precisamente, no Firestore Database a estrutura será a seguinte:



```
{
  "Document ID": "CTE80ZCSueT2DbuonNkel21MHDj2",
  "favoriteCharacters": "[{id: 3015, name: \"Jim Hawk...\", {name: \"Esmeralda\", imageU...}, {id: 4703, name: \"Mickey M...}]",
  "favoriteFilms": "[{title: \"Treasure Planet\",...}, {title: \"Mickey, Donald, G...}]",
  "profileImage": "/9j/4AA...uS//2Q== "
}
```

Figura 4 - Estrutura do User no Firestore Database

O User tem um ID, uma lista de personagens favoritos, uma lista de filmes favoritos e uma foto de perfil.

A foto de perfil é guardada em base64, isto porque de outra forma teria de ser utilizado o Firebase Storage que é pago, devido a isso, optou-se por passar a imagem para base64 e assim fica tudo centralizado no Firestore Database do Firebase.

A lista de personagens favoritos também uma estrutura própria. Cada personagem é composto de:

- **id** - Valor id vindo da Disney API que identifica o personagem. Isto serve para o caso de personagens que têm mais do que uma versão, ou personagens diferentes, mas que têm o mesmo nome.
- **imageUrl** - Campo que possui o link direto para a imagem do personagem
- **name** - Campo que possui o nome do personagem

Na lista de filmes favoritos, cada filme é composto de:

- **imageUrl** - Campo que possui o link direto para o poster do filme
- **title** - Campo que possui o nome do filme

As listas de filmes criadas pelo utilizador são uma coleção à parte das que já foram apresentadas, contudo continuam associadas ao utilizador. A estrutura destas listas é a seguinte:



Figura 5 - Estrutura das listas criadas pelo User

O `id` serve para identificar a lista, o campo `films` é composto por vários filmes com a estrutura apresentada anteriormente. O campo `listName` apresenta o nome da lista, o `selectedStatus` apresenta o valor que o utilizador selecionou na *dropbox* e o `watchedCount` é o número de filmes que o utilizador indica o número de filmes que já viu da lista.

Produção - Implementação

Sistema de Registo e Login

A aplicação foi separada em três módulos: `api`, `ui` e `view Model`.

Começando por abordar o sistema de registo do utilizador, este é composta por um `Scaffold` (que é como se fosse um *layout* básico que suporta várias *features* nele). Dentro deste componente temos uma `TopBar` do tipo `Large`.

Em seguida, temos na página em si (na função do `ScrollContent` que vai ser chamada na parte de `inner padding` do `Scaffold`) com uma coluna. Esta coluna é composta por quatro campos:

- **Nome:** Campo de texto para inserir o nome do utilizador.
- **Email:** Campo de texto para inserir o endereço de email, que será utilizado para autenticação.
- **Password:** Campo de texto com transformação visual para ocultar a password durante a digitação.
- **Confirmar Password:** Campo repetido para validação da password, assegurando que o utilizador introduziu a password corretamente.

Quando se clica no botão `Create`, o método `createUserWithEmailAndPassword` do `Firebase Auth` é chamado para registar o utilizador com o email e password fornecidos (já depois da devida configuração feita no `Firebase`). Se a operação no `Firebase` for bem-sucedida, o utilizador é autenticado e o seu nome é atualizado no perfil através de outro método do `Firebase` chamado `UserProfileChangeRequest`.

Em caso de sucesso, é apresentada uma mensagem de sucesso e o utilizador é redirecionado para a página de login. No caso do registo falhar, é exibida uma mensagem de erro com detalhes do ocorrido.

Caso o utilizador já possua uma conta, pode optar por ir para a página de login através do hyperlink "*Already have an account? Login*", que também remove a página de registo da pilha de navegação.

Quanto ao sistema de login, assim como em todos os ecrãs realizados para este projeto, tem um `Scaffold` e uma `LargeTopBar` por base. Os componentes do ecrã também são organizados numa coluna, onde são postos três campos de texto para o Nome, Email e Password respetivamente.

Para a autenticação do Login, usa-se o método `signInWithEmailAndPassword` do `Firebase Auth`. Se a

autenticação for bem-sucedida, o utilizador é redirecionado para a página principal. São apresentadas mensagens de sucesso ou falha baseado no estado da autenticação.

Semelhante ao que já havia no ecrã de registo, também temos um hyper link, mas neste caso é para que quando o utilizador ainda não tenha uma conta seja redirecionado para a página de registo.

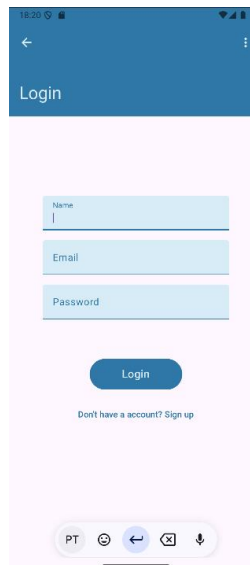
A mobile app login screen with a blue header bar containing a back arrow and the word 'Login'. The main area is light pink and contains three light blue input fields for 'Name', 'Email', and 'Password'. Below these is a blue 'Login' button and a link that says 'Don't have an account? Sign up'. At the bottom is a white navigation bar with icons for 'PT', a home icon, a back arrow, a search icon, and a download icon.

Figura 6 - Ecrã de Login

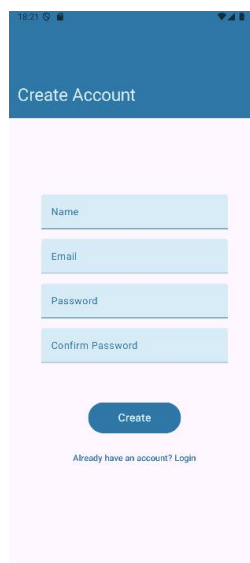
A mobile app 'Create Account' screen with a blue header bar containing the text 'Create Account'. The main area is light pink and contains four light blue input fields for 'Name', 'Email', 'Password', and 'Confirm Password'. Below these is a blue 'Create' button and a link that says 'Already have an account? Login'. At the bottom is a white navigation bar with icons for 'PT', a home icon, a back arrow, a search icon, and a download icon.

Figura 7 - Ecrã de Criar Conta

Navegação

Visto que se abordou a navegação entre os ecrãs. Isto é feito recorrendo ao componente NavHost do Jetpack Navigation.

O objeto principal nesta parte do projeto é o navcontroller que é um rememberNavController, que permite controlar as transições entre os ecrãs. Este controlo baseia-se num “buffer” de ecrãs onde ficam guardados os ecrãs visitados e assim permite-nos usar o popBackStack para voltar a um ecrã anterior.

Começamos por verificar o estado de autenticação do *user* com o LaunchedEffect (que executa corrotinas quando o ecrã é inicializado ou quando os seus parâmetros mudam. Neste caso o parâmetro é um booleano que

indica se o utilizador está ou não autenticado.

Se o utilizador estiver autenticado, redireciona-o automaticamente para o ecrã principal ("home") e elimina a rota do login do buffer para evitar retrocessos indesejados.

Cada ecrã é definido como um `composable`, associado a uma rota específica (por exemplo, "home", "login", "profile"). Algumas rotas têm parâmetros dinâmicos como é o caso do `listName` e do `movieTitle`, que possuem parâmetros codificados e decodificados para garantir compatibilidade com os links que vêm da API.

API Client

Aplicação fundamenta-se quase por completo nos pedidos que faz às APIs já referidas. Para realizar estes pedidos criou-se um cliente HTTP com a ajuda da biblioteca Ktor.

Para a configuração deste cliente usou-se o plugin `ContentNegotiation` para tratar da serialização/desserialização de JSON, configurado para ignorar *keys* desconhecidas, permitir JSON leniente e omitir valores nulos.

Foram criadas classes serializáveis que mapeiam a estrutura dos dados recebidos da API, para ser mais fácil obter os campos de interesse da resposta JSON com os dados desejados.

Aqui também são definidas as funções para obter os posters e informação dos filmes e a informação sobre os personagens.

Para obter os posters, a função `fetchMoviePoster` recebe o título do filme e faz um GET à API da WikiFandom da Disney, e retorna o valor do campo `thumbnail` que é o link direto para o poster do filme.

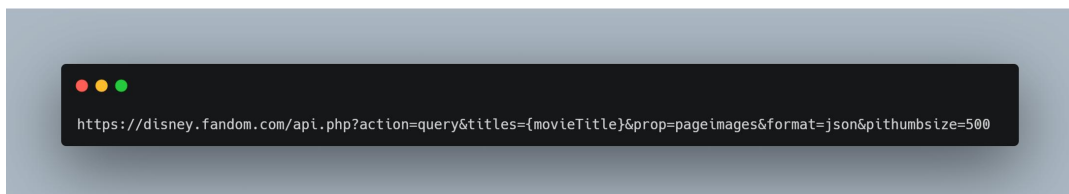


Figura 8 - Endpoint para o obter os posters.

Para obter a informação dos filmes (runtime e data de lançamento), não existia nenhum endpoint direto para obter estas informações nas duas APIs. Portanto a solução encontrada, embora tenha falhas devido à formatação variar de filme para filme, foi retirar estes dados diretamente do HTML. Tendo o HTML da página, são utilizadas expressões regulares para obter os campos desejados. Tudo isto é feito na função `fetchMovieDetails`.

Por fim, para termos os dados sobre os personagens recorremos à função `fetchCharacters`, que é usada numa barra de pesquisa do ecrã principal. Por isso `query` é feita com o input do jogador e suporta um sistema de paginação presente na API. Cada resposta é desserializada para disponibilizar de forma mais simples o nome, o URL da imagem de perfil dele, o seu id e os filmes em que participa.

Todas as funções de `fetch` são `suspend`, para garantir de que os pedidos são assíncronos e não bloqueiam a

thread principal.

Ecrã Principal

O ecrã principal da aplicação tem apresentadas várias listas, onde cada uma representa uma era da Disney. É implementada uma arquitetura Model-View-ViewModel (MVVM) para gerir o estado dos dados e a lógica de negócio.

A Scaffold deste ecrã tem dois modos: um modo padrão onde são exibidas as listas das diversas eras da Disney e outro modo que é o modo de pesquisa de personagens. Quando as listas estão presentes temos a estrutura baseada numa LazyColumn, que por sua vez é composta por carrosséis com a estrutura definida na função DisneyCarouselHome.

O HomeViewModel fornece os dados dos filmes organizados pelos períodos históricos e usa a função fetchMoviePosters do APIClient para obter os posters a partir dos títulos, que neste caso já estão pré-definidos. Este ViewModel ao ser inicializado começa logo por fazer o fetch dos posters das várias eras.

Já o CharacterSearchViewModel para da procura dos personagens, que é feita através do fetchCharacters já mencionado. Para evitar um envio exagerado de pedidos à API é aplicado um delay de 300ms.

Enquanto os posters carregam ou no caso de não carregarem é posta no lugar uma imagem local.

A partir do ecrã principal dá para ir para a página de perfil do *user*, para o perfil de um filme (clicando num poster), o perfil de um personagem clicando no nome quando pesquisado e para uma lista quando se clica na seta presente sobre ela.

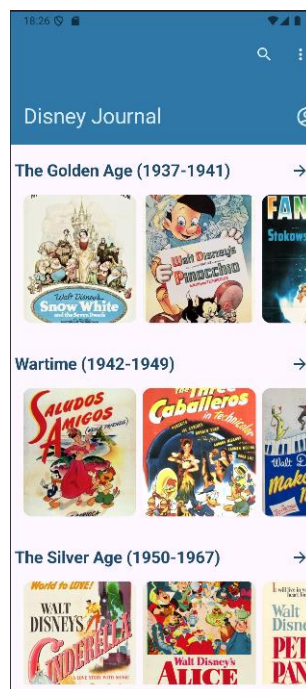


Figura 9 - Ecrã Principal

Lista de Filmes

Este ecrã permite ao utilizador visualizar, gerir e acompanhar o progresso de visualização dos filmes de uma lista. A interface integra-se com o Firebase Firestore para persistência de dados à medida que o *user* os vai atualizando.

Podemos dividir este ecrã em duas secções: uma de estatísticas sobre lista e outra com a lista em si. Cada lista (retirando as das Eras da Disney) Cada lista é armazenada na subcoleção myLists do utilizador autenticado (`users/{uid}/myLists`).

Na secção de estatística existe um campo de texto numérico (TextField) para inserir o número de filmes vistos, que aceita apenas valores numéricos dentro do limite total de filmes. Também existe uma dropdown para o *status* da lista com as opções: "Watching", "Dropped", "On Hold", "Plan to Watch". Por último, é apresentada uma barra de progresso que mapeia o número de filmes vistos sobre o número total de filmes da lista.

É possível também apagar a lista, se o icon de dele da TopBar for pressionado, ou apagar apenas um filme ao clicar num icon semelhante apresentado no respetivo listItem.

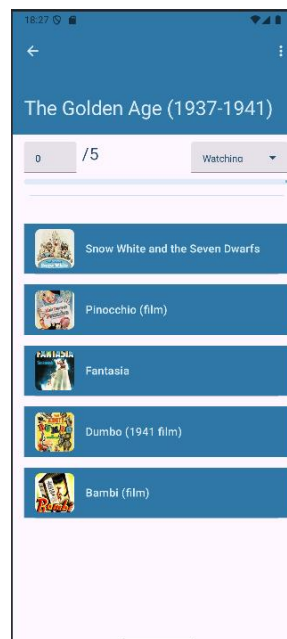


Figura 10 - Ecrã com Lista de Filmes

Perfil de Filmes e Personagens

Estes ecrãs são muito semelhantes, divergem apenas no facto que se o ecrã for de personagem, sob o card com a foto dele, teremos uma lista dos filmes dos quais este participa e se for o perfil de um filme teremos a informação (tempo de duração e data de lançamento).

Na TopBar temos o nome do personagem/filme e uma estrela encostada à direita para o utilizador favoritar. No Firestore os favoritos são guardados numa lista favoriteCharacter e favoriteFilms que já foram apresentadas acima. Ao serem desmarcadas, o elemento é automaticamente removido da lista.

No ecrã de persongaem, ao clicar num filme que este participa somos redirecionados para o perfil desse mesmo filme.

No ecrã do perfil do filme é apresentado um botão de “Add to List” que ao clicarmos nele é exibido uma caixa de diálogo com lista de checkboxes onde cada caixa representa uma lista criada pelo utilizador. Ao marcar as listas desejadas e clicar em Ok, estes filmes são adicionados a uma coleção do Firestore com os respetivos dados.

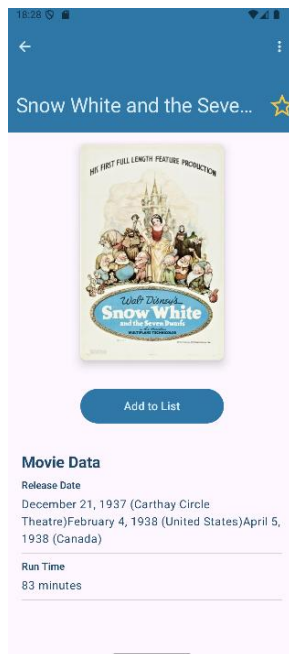


Figura 11 - Ecrã do Perfil do Filme

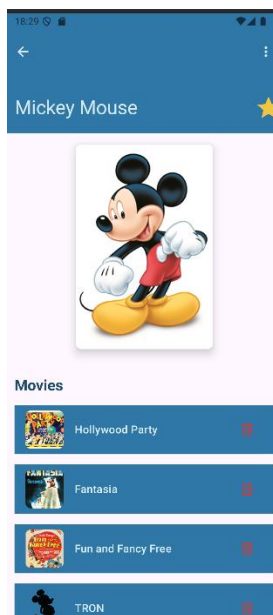


Figura 12 - Ecrã do Perfil do Personagem

Perfil do utilizador

O ecrã de utilizador possui um botão para editar a foto de perfil, o nome do *user* e o botão para fazer logout que está presente em todos os ecrãs. Abaixo da TopBar temos um avatar com a foto de Perfil e sob ela um botão

para criar lista (ao clicar nele surge uma caixa de diálogo, `CreateListDialog`, para inserir o nome da lista criar).

Sob este botão temos carrosséis idênticos aos já mostrados, mas cada um tem uma finalidade:

- Personagens favoritos
- Filmes favoritos
- Listas criadas

As listas criadas são associadas ao user ao serem guardadas sobre o seu id: `users/{uid}/myLists`.

Para uma melhor percepção de como os dados estão estruturados e associados entre si, segue um esquema simples:

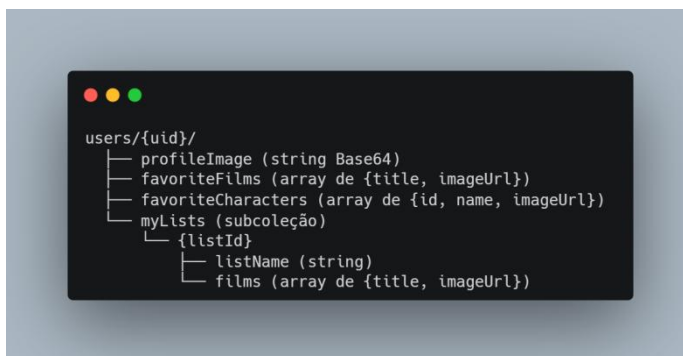


Figura 13 - Estrutura dos dados do Firebase, associados ao user

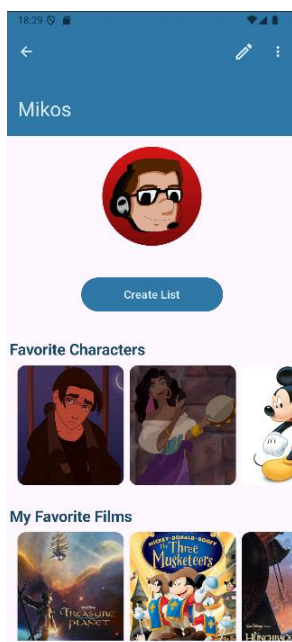


Figura 14 - Ecrã do Perfil do User

Favoritar Filmes e Personagens

O sistema de favoritos na aplicação Disney Journal permite aos utilizadores marcar os seus filmes e personagens preferidos, criando uma coleção personalizada que pode ser acedida rapidamente a partir do seu perfil. Esta funcionalidade foi implementada utilizando o Firebase Firestore para guardar os dados de forma persistente e associá-los à conta do utilizador.

Os favoritos são armazenados no Firestore em duas listas distintas dentro do documento do utilizador, como mostrado na imagem anterior.

A lista dos filmes favoritos possui como campos de cada participante o título e o URL da imagem do poster e a lista dos personagens possui o nome, o id e a imagem que dizem respeito a cada personagem favorito.

Em ambos os casos, a ação de favoritar é realizada da mesma forma: clicando no ícone da Estrela Amarela eu fica na TopBar dos respetivos ecrãs.

Quando o utilizador clica no ícone, a função `toggleFavorite()` é invocada, e começa o processo de favoritar que se divide em três etapas: verificação de autenticação, preparação dos dados e atualização dos dados no Firestore.

Primeiro, o sistema recupera a lista atual de filmes favoritos do documento do utilizador. Em seguida, consoante o estado atual, ou adiciona um novo registo (com o título do filme e URL da imagem) ou remove um existente, caso o título do filme exista na lista (o que acontece quando de desseleciona a estrela).

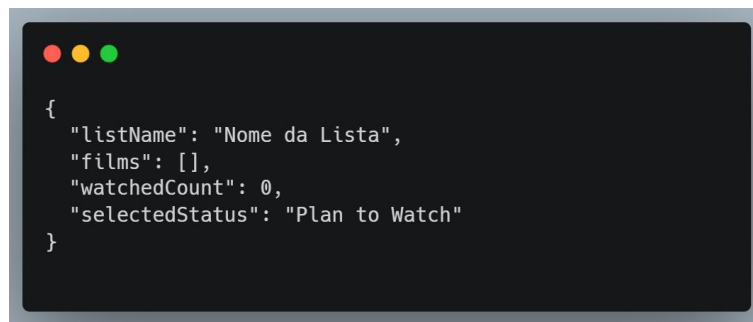
Foi implementado um *listener* (`SnapshotListener`) que monitoriza continuamente as alterações na coleção de favoritos. Este listener, ativado durante a inicialização do ecrã, assegura que qualquer modificação na base de dados seja imediatamente refletida na interface do utilizador. Assim, por exemplo ao selecionar um filme ou personagem como favorito, este fica logo com a estrela pintada e a base de dados no Firebase ganha uma entrada na lista respetiva ao elemento favoritado.

Os filmes e personagens favoritos ficam disponíveis na página do perfil do utilizador, apresentados em dois carrosséis diferentes. Ao clicar num filme ou num personagem que esteja como favorito, somos redirecionados para respetiva página desse elemento.

Criar Listas

Outra funcionalidade fundamental na *app* Disney Journal é a criação de listas e a adição de filmes às mesmas.

As listas são guardadas numa coleção própria associadas ao user que as cria. Todas as listas seguem a seguinte estrutura:



```
{
  "listName": "Nome da Lista",
  "films": [],
  "watchedCount": 0,
  "selectedStatus": "Plan to Watch"
}
```

Figura 15 - Estrutura das listas criadas pelo utilizador

O fluxo de criação de uma lista começa ao clicar no botão de “Create to List” que abre um *modal* com um campo para inserir o nome da lista. A partir deste momento o carrossel das listas criadas ganha uma entrada e é logo apresentado o nome da lista e quantos filmes tem associados.

Ao clicar na lista, somos redirecionados para o ecrã da lista em si, com a contagem de filmes, e o *status* dela.

Para adicionar um filme a uma lista é necessário ir até a página de perfil do filme e clicar no botão “Add to list” que abre um *modal* com uma *checkbox* para cada lista criada. Ao clicar no botão “Add” guarda-se o nome do filme e o URL da imagem do póster. Depois estes dados são adicionados na lista do utilizador no Firebase com a ajuda da função `FieldValue.arrayUnion()` para evitar duplicações na lista.

Mudar Cartazes dos Filmes

Por fim, implementou-se a segunda funcionalidade para utilizadores que pagaram o plano *premium*. Esta funcionalidade é a possibilidade de trocar de cartaz de uma filme na sua página de perfil.

Ao pressionar o cartaz do filme, surge um *modal* com três opções: Select from Gallery, Set Default e Cancel. Ao clicar na primeira opção, abre-se uma janela da galeria para o *user* escolher a foto do poster. Depois converte-se a imagem para Bitmap e depois para base64 para a guardar no Firebase.

Com isto o user fica com mais uma lista a seu cargo no Firebase: `customPoster`, onde cada participante tem o título do filme e a imagem em base64 associados.

Ao renderizar o ecrã do perfil do filme, verifica-se se este tem um póster personalizado. Caso tenha este é apresentado, caso contrário é apresentado o cartaz padrão do filme que vem da Wiki.

Se o utilizador no *modal* escolher o Set Default, o póster é eliminado do Firebase e volta-se a apresentar o póster padrão do filme.

Conclusão

Cumpriu-se com tudo o que era suposto implementar na *app* proposta, embora com alguns problemas, devido à falta de suporte das APIs usadas e da variação da estrutura das páginas HTML e também das APIs com dados dos personagens e dos filmes.

Mesmo assim, o desenvolvimento deste trabalho trouxe um maior conhecimento em como trabalhar com dados vindos de uma API que não tinham ficado tão bem retidos noutros trabalhos e o facto de fazer a aplicação toda do zero trouxe um maior domínio sobre a mesma, o que é bastante gratificante.

Para trazer maior realismo na apresentação da aplicação, as *features premium* poderiam ter um modal a barrá-las para simular uma *paywall*. Contudo, para uma maior facilidade na apresentação do trabalho, optou-se por não colocá-la.

Como já foi mencionado, ainda haveria espaço para algumas melhorias, mas penso que o resultado final da aplicação, na qual se conseguiu implementar todas as funções propostas, esteja bastante satisfatório.

Bibliografia

stplbloggersouthslidell. (2024, December 9). *St. Tammany Parish Library*. Sttammanylibrary.org.
<https://www.sttammanylibrary.org/blogs/post/disney-movie-eras-the-golden-age/>

(2023) . Disneyapi.dev. <https://disneyapi.dev/>

MediaWiki API help | Disney Wiki | Fandom. (2020). Fandom.com. <https://disney.fandom.com/api.php>

Philipp Lackner. (n.d.). YouTube. Retrieved June 22, 2025, from
<https://www.youtube.com/channel/UCKNTZMRHPLXfqldOI7mCkg>

Android Knowledge. (2025, May 12). *Login & Signup using Firebase Authentication + Jetpack Compose Android Studio | The JetBase Series*. YouTube. <https://www.youtube.com/watch?v=vI3pK4JvZzY>